US Communities Crime Explorer

Individual Assignment — Interactive Data Visualizations in React and D3.js

1. Characterization of the Problem, Data, and User Tasks

Problem. The user's objective is to decide in which US state or city to settle down to live. Crime rate in relation to income level and community size are the key variables affecting this decision.

Data. The UCI Communities and Crime dataset (`https://archive.ics.uci.edu/dataset/183/communities+and+crime`) describes 1,994 US communities using 128 attributes, all normalized to $[0, 1]$. Attributes used: *ViolentCrimesPerPop*, *medIncome*, *population*, *state* (integer 1–56), and *communityname*.

User tasks and application responses.

- **Identify safer and more populous communities** — crime rate is encoded by **color** (red = high, green = low) and population by **size** in both the Treemap and Sunburst; crime rate is also readable by **vertical position** in the scatterplot.
- **Correlate income with crime** — the scatterplot (x = income, y = crime) makes the negative correlation immediately visible as a spatial pattern.
- **Drill down and filter** — clicking a state node selects all its communities and highlights them in the scatterplot; the 2D brush on the scatterplot does the reverse, selecting a cluster and highlighting it in the hierarchy.
- **Identify communities by name** — hovering any dot or node shows a tooltip with community name, state, and normalized values; a selection info bar lists selected names grouped by state.

Figure 1: Treemap: filtered states and communities with high income and low crime rate.

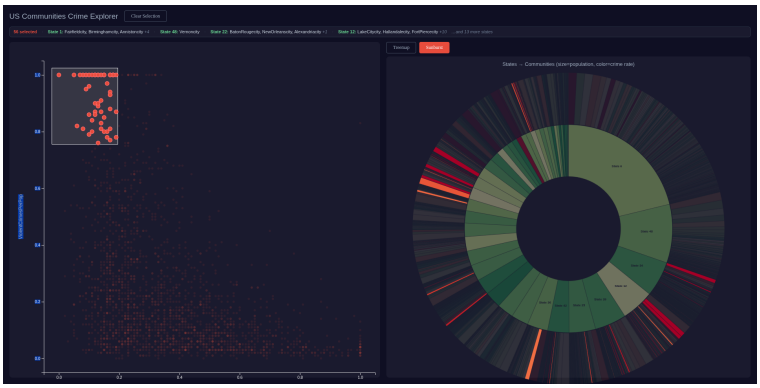

Figure 2: Sunburst: filtered states and communities with high crime and low income.

2. Design Rationale

Scatterplot. Position is the most perceptually accurate visual channel, making the scatterplot the natural choice to reveal the income–crime correlation. The **2D brush** (`d3.brush`) lets the user drag a rectangle to select communities; non-selected dots dim to 0.1 opacity and the hierarchy reacts simultaneously. Hovering shows a tooltip — progressive disclosure with zero permanent screen cost.

Hierarchical layouts. Both layouts group 1,994 communities under their state, encoding crime by **color** and population by **size**. The **Treemap** uses nested rectangles: space-filling, all communities visible at once, and rectangular area is perceptually more accurate than arc angle for comparison. Its only limitation is that very small communities produce thin rectangles. The **Sunburst** uses concentric arcs and is visually appealing, but with nearly 2,000 communities many outer arcs become too thin to click or label, and angular distortion makes area comparisons less precise.

Preferred layout: Treemap. The primary task is attribute-based comparison — finding communities with low crime and large population. Per the course lectures (Ghoniem, MHPC 2026), the Treemap is the recommended technique for *quantitative proportions*; its space-filling property, accurate area channel, and readable labels make it the more effective choice for this dense, attribute-rich dataset.

Synchronized interactions. Both views share a Redux store (`ItemInteractionSlice`). Brushing the scatterplot or clicking a hierarchy node dispatches `setSelectedItems`; a `useEffect` in each container reacts simultaneously with animated transitions. Ctrl+Click adds to an existing selection without replacing it.

3. Declaration of Generative AI Use

Model used: Claude Sonnet (Anthropic), via `claude.ai`. All prompts and responses are in *prompts_and_ai_answers.pdf* at the root of the repository.

What AI did. All implementation code was generated with AI assistance: the D3 classes, React containers, Redux store and slices, and CSS files. A first draft of this report was also AI-generated and subsequently edited.

What I decided myself. All visualization design decisions were made independently: the scatterplot for income–crime correlation, the two-level hierarchy, the comparison of Treemap and Sunburst, and the preference for the Treemap. These judgments required understanding the user's task and visualization technique properties — they cannot be delegated to a code generator.

What I rejected or fixed. Two significant bugs required manual diagnosis. First, `clearBrush()` caused an infinite loop fixed by adding a `_programmaticClear` flag. Second, storing 128-column objects in Redux froze the browser on large selections, fixed with `serializableCheck: false` in `store.js`. The missing CSV header row was also resolved manually via a Python script.

Dataset: UCI Communities and Crime — https://archive.ics.uci.edu/dataset/183/communities+and+crime